Software Requirements and Design Document

For

Group <5>

Project

RaceTrack

Version 3.0

Authors:
Olivia Reiter
Julissa Su
Brooklyn Metzger
Yulissa Fu
Aleksandar “The Great” Stavreski
Alexander Hsieh

1. Overview (5 points)

Give a general overview of the system in 1-2 paragraphs (similar to the one in the project proposal).

RaceTrack is a web application that leverages machine learning to predict Formula 1 race outcomes using historical performance data, weather conditions, and race strategy parameters. The system provides users with race winner probability predictions for upcoming races, current driver and constructor championship standings, and a customizable race simulation tool that allows adjustment of variables such as circuit selection, weather conditions, and driver grid selection. The application also includes a comprehensive historical data center covering race results from 2010 to present, and a newsroom aggregating current F1 updates from multiple sources.

The system is built on a React/TypeScript frontend, a Python FastAPI backend, and a PostgreSQL database, with machine learning components developed in Python using scikit-learn (RandomForestClassifier, 12-feature input vector including real weather data). It is designed as an open-access platform requiring no user authentication, prioritizing ease of use and data transparency over competitive or social features — distinguishing it from existing F1 prediction apps that focus primarily on fantasy leagues and point-based competitions. To keep predictions current across the active season, the system incorporates an automated data ingestion pipeline powered by APScheduler that pulls and stores 2026 race results as they become available, without requiring manual model retraining. The frontend and backend are both deployed on Render, with the React application served as a static site and the FastAPI backend running as a web service.

2. Functional Requirements (10 points)

*List the **functional requirements** in sentences identified by numbers and for each requirement state if it is of high, medium, or low priority. Each functional requirement is something that the system shall do. Include all the details required such that there can be no misinterpretations of the requirements when read. Be very specific about what the system needs to do (not how, just <u>what</u>). You may provide a brief design rationale for any requirement which you feel requires explanation for how and/or why the requirement was derived.*

- **FR-1: ML Prediction Service (High Priority)** The system shall predict race winner probabilities for a given race using a trained machine learning model (RandomForestClassifier v3.0, serialized as f1_winner_model_v3.pkl), targeting 45–50% exact top-1 winner accuracy and 70% or greater top-3 podium prediction accuracy on held-out test data.
- **FR-2: Feature Calculation (High Priority)** The system shall calculate a 12-feature input vector per prediction request, comprising driver statistics (win rate, average finish, podium rate, recent form), team performance average, circuit-specific driver performance, qualifying position delta, grid position, and normalized real weather data (temperature, humidity, wind speed, rainfall intensity).
- **FR-3: Model Versioning (Medium Priority)** The system shall track and log the ML model version currently in use, accessible via backend configuration or API response metadata.
- **FR-4: Dashboard Display (High Priority)** The system shall display upcoming F1 race information on the Dashboard, including race name, circuit location, a live countdown to the next scheduled race, and a real-time dual clock showing MY TIME and TRACK TIME derived from the race circuit's IANA timezone.

- **FR-5: Real-time UI Updates (Medium Priority)** The frontend Dashboard shall automatically refresh race data every five minutes in the background without requiring a full page reload.
- **FR-6: API Client Data Retrieval (High Priority)** The system shall retrieve F1 data — including next race, circuit details, and standings — through a centralized API client layer in the frontend, with retry logic for failed requests.
- **FR-7: Historical Race Data Retrieval (High Priority)** The system shall fetch historical F1 race schedules, results, driver standings, and constructor standings from the Jolpica F1 API.
- **FR-8: Upcoming Race Schedule (High Priority)** The system shall expose an endpoint that returns the next scheduled race and the race immediately following it, used to populate the Dashboard race cards.
- **FR-9: Database Population (High Priority)** The system shall populate the database with race schedules, circuit information, driver and team records, and a race calendar containing start datetime, end datetime, and sprint race flag for all 2026 rounds, preventing duplicate entries through validation checks on year and round number combinations.
- **FR-10: Idempotent Seeding (Medium Priority)** The system shall allow database seeding scripts to be run multiple times safely, updating existing records rather than creating duplicates.
- **FR-11: Race Simulation (High Priority)** The system shall provide a Simulator page that accepts user-defined parameters — including circuit, weather condition, and starting grid order — and returns a predicted race result by invoking the trained ML model via the /api/simulator/ endpoint using live model inference. The simulator shall resolve the selected circuit to a valid 2026 race_id from /api/races to ensure modern driver IDs are accepted without error.
- **FR-12: Data Center Page (Medium Priority)** The system shall provide a Data Center page that allows users to browse historical race results, circuit information, and driver/constructor standings, with SVG circuit visualizations for active F1 venues.
- **FR-13: Newsroom Page (Medium Priority)** The system shall provide a Newsroom page that aggregates current F1 news from multiple external RSS sources via feedparser and displays articles in a filterable card-based layout.
- **FR-14: Automated Data Ingestion Pipeline (High Priority)** The system shall automatically pull and store 2026 race results from the Jolpica F1 and OpenF1 APIs as each race completes, using APScheduler to trigger ingestion jobs approximately two hours after each race end datetime, updating the database via idempotent upsert functions without requiring manual intervention or model retraining.
- **FR-15: Driver Grid Access (High Priority)** The system shall provide a GET /api/drivers endpoint that returns current-season drivers with their respective team colors and nationality flags, used to populate the Simulator starting grid.
- **FR-16: Weather Service Roles (High Priority)** The system shall use Visual Crossing for one-time historical weather backfills used in ML model training, and OpenWeatherMap for ongoing live and forecast weather data surfaced in the Simulator and Dashboard.
- **FR-17: Driver Standings Database Write (High Priority)** The system shall upsert driver standings into the driver_standings table from normalized API data, updating position, points, and win_chance on conflict with an existing record for the same year and driver.
- **FR-18: Constructor Standings Database Write (High Priority)** The system shall upsert constructor standings into the team_standings table from normalized API data, updating

position, points, and win_chance on conflict with an existing record for the same year and team.

- **FR-19: Race Results Database Write (High Priority)** The system shall insert race results into the race_results table with the following fields: race_id, circuit_id, driver_id, team_id, grid_position, finish_position, points_scored, race_date, and weather_condition, updating existing records on conflict with the same race_id and driver_id combination.
- **FR-20: Cache Invalidation on Write (High Priority)** The system shall invalidate cached API responses for driver standings, constructor standings, next race, and upcoming races endpoints after each successful database write operation.
- **FR-21: Constructor Standing Team Colors (Medium Priority)** The system shall return constructor standings with a team color hex code for each constructor, sourced from a predefined team color mapping covering all 10 active 2026 constructors.

3. Non-functional Requirements (10 points)

*List the **non-functional requirements** of the system (any requirement referring to a property of the system, such as security, safety, software quality, performance, reliability, etc.) You may provide a brief rationale for any requirement which you feel requires explanation as to how and/or why the requirement was derived.*

- **NFR-1: Prediction Performance (High Priority)** The system shall generate race predictions and return a response to the frontend in under 2 seconds under normal operating conditions, excluding Render cold start latency on the free tier.
- **NFR-2: Model Reliability (High Priority)** The system shall handle ML model loading failures gracefully, returning a clear error message to the frontend rather than crashing the backend service.
- **NFR-3: Model Accuracy (High Priority)** The ML model shall achieve a minimum of 45% top-1 winner prediction accuracy and 70% top-3 podium prediction accuracy on held-out test data. These targets reflect the inherent unpredictability of F1 race outcomes and are intentionally conservative to ensure defensible, reproducible results.
- **NFR-4: Usability / Responsiveness (Low Priority)** The frontend interface shall be responsive, maintaining a readable and functional layout on both desktop (1920×1080 and above) and mobile screen sizes, with layout scaling handled via CSS media queries rather than browser zoom.
- **NFR-5: Frontend Reliability (High Priority)** The frontend API client shall gracefully handle network errors, retrying failed requests up to three times before displaying a user-facing error message.
- **NFR-6: API Rate Limiting (High Priority)** The system shall enforce a rate limit of no more than 2 calls per second to external APIs to prevent service abuse and comply with third-party usage policies.
- **NFR-7: Seeding Performance (Medium Priority)** The system shall complete database seeding of one full F1 season's worth of data in under 60 seconds on standard development hardware.
- **NFR-8: Cache Storage Efficiency (Low Priority)** The system shall store cached API responses in a structured file-based format that minimizes redundant external calls across application restarts.
- **NFR-9: Data Ingestion Reliability (Medium Priority)** The automated data ingestion pipeline shall handle API failures and partial responses without corrupting existing database records, logging errors for manual review when ingestion cannot complete successfully.

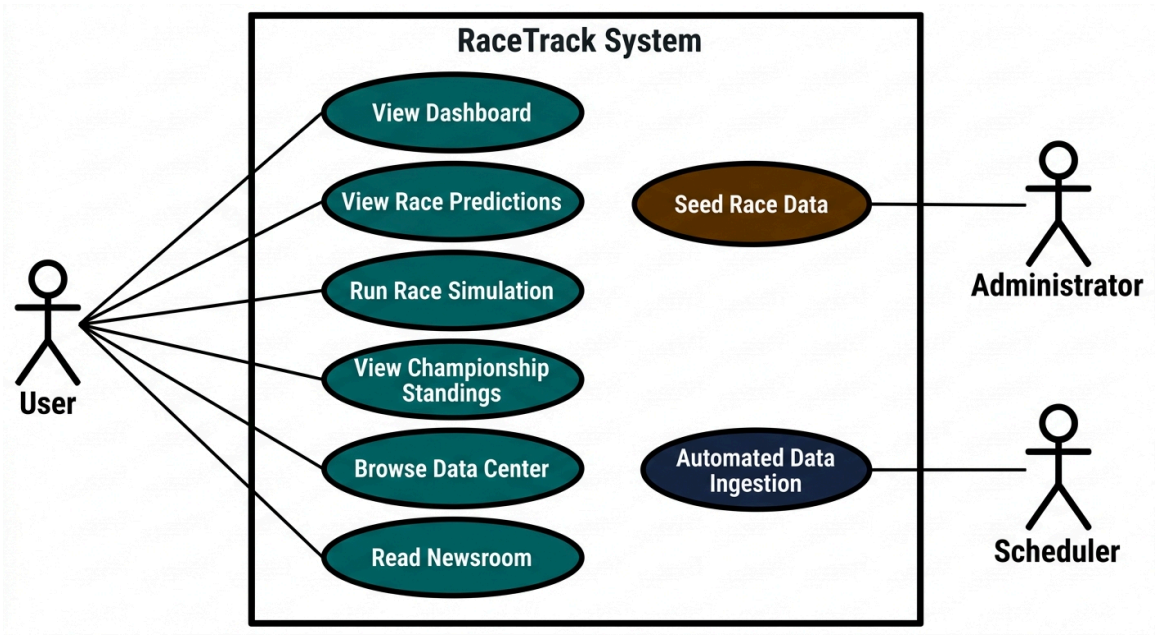
- **NFR-10: Memory Management (Medium Priority)** The frontend shall implement proper cleanup by clearing intervals on component unmount to prevent memory leaks from the real-time clock and background refresh timers.
- **NFR-11: Timezone Awareness (Medium Priority)** The system shall derive TRACK TIME using a country-to-IANA-timezone mapping with `Intl.DateTimeFormat` for DST-aware calculations, falling back gracefully when timezone data is unavailable from the API.
- **NFR-12: Database Write Logging (Medium Priority)** The system shall log the number of rows inserted or updated per table after each successful database write operation to support debugging and auditing of the automated data ingestion pipeline.
- **NFR-13: Production Deployment Stability (High Priority)** The backend shall bind to the host and port provided by the Render environment at startup, and the frontend shall be served as a static site with a `_redirects` rule ensuring all routes resolve to `index.html` for client-side routing to function correctly in production.

4. Use Case Diagram (10 points)

*This section presents the **use case diagram** and the **textual descriptions** of the use cases for the system under development. The use case diagram should contain all the use cases and relationships between them needed to describe the functionality to be developed. If you discover new use cases between two increments, update the diagram for your future increments.*
*Textual descriptions of use cases: **For the first increment, the textual descriptions for the use cases are not required.** However, the textual descriptions for all use cases discovered for your system are required for the second and third iterations.*

Use Case Diagram

This diagram outlines the discrete actions available to Users, Administrators, and the automated Scheduler within the RACETRACK system boundary.



5. Class Diagram and/or Sequence Diagrams (15 points)

*This section presents a high-level overview of the anticipated system architecture using a **class diagram** and/or **sequence diagrams**.*

*If the main **paradigm** used in your project is **Object Oriented** (i.e., you have classes or something that acts similar to classes in your system), then draw the **Class Diagram of the entire system and Sequence Diagrams for the three (3) most important use cases in your system.***

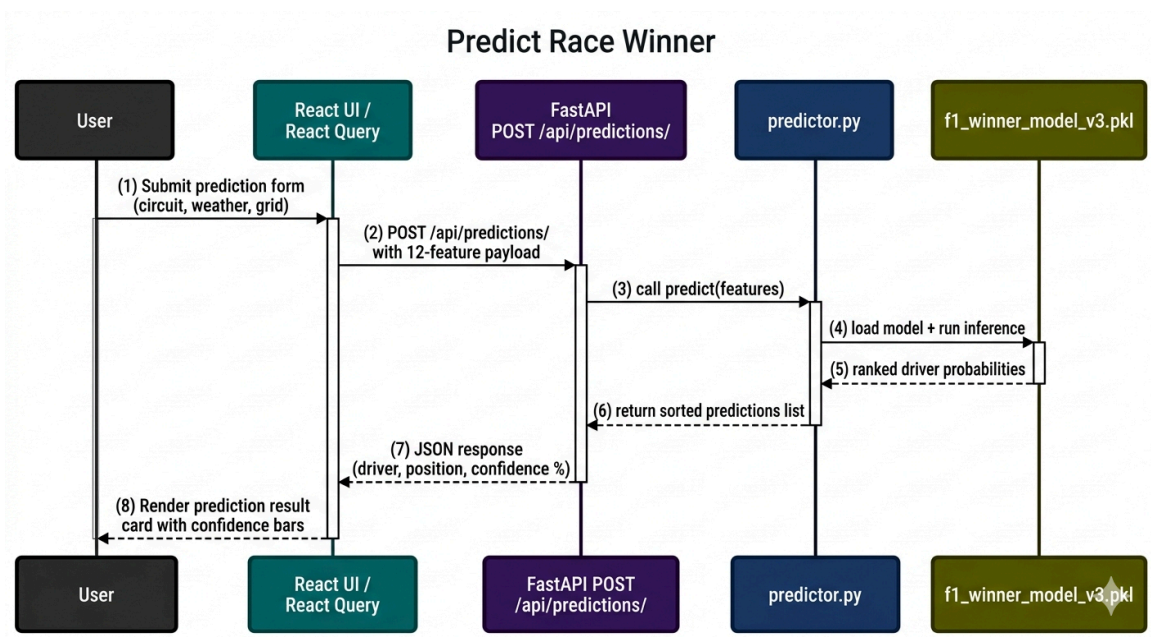
*If the main **paradigm** in your system is **not Object Oriented** (i.e., you **do not** have classes or anything similar to classes in your system) then only draw **Sequence Diagrams**, **but for all the use cases of your system.** In this case, we will use a modified version of Sequence Diagrams, where instead of objects, the lifelines will represent the functions in the system involved in the action sequence.*

***Class Diagrams** show the **fundamental objects/classes** that must be modeled with the system to satisfy its requirements and **the relationships** between them. Each class rectangle on the diagram **must also include the attributes and the methods of the class** (they can be refined between increments). All the **relationships between classes and their multiplicity** must be shown on the class diagram.*

*A **Sequence Diagram** simply depicts **interaction between objects** (or **functions -** in our case - for non-OOP systems) in a sequential order, i.e. the order in which these interactions take place. Sequence diagrams describe how and in what order the objects in a system function*

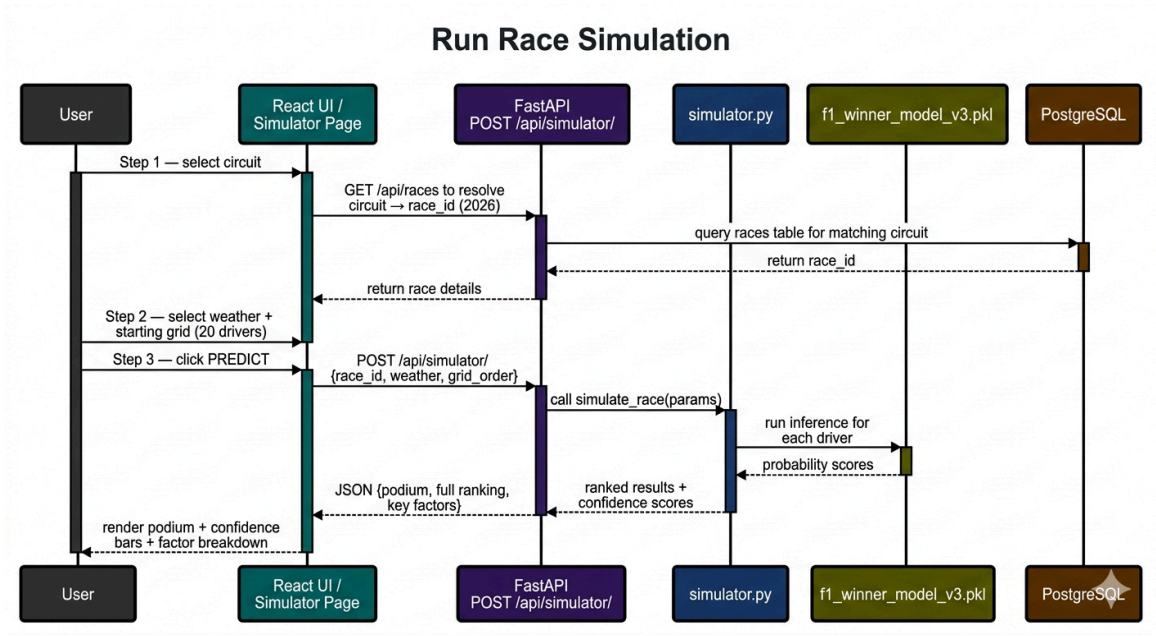
Sequence Diagram — Predict Race Winner

This diagram details the high-contrast flow for generating a single race prediction. Crucially, it shows the active `predictor.py` and the `v3.pkl` model, replacing the previous hardcoded stub.



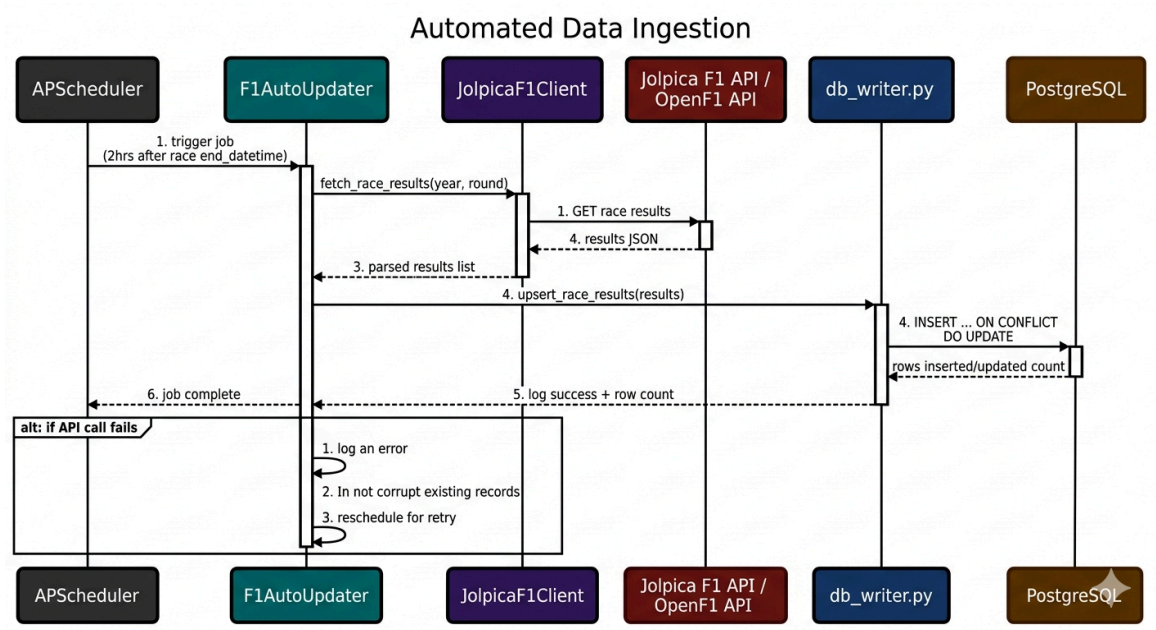
Sequence Diagram — Run Race Simulation

This new diagram details the interactive 3-step Simulator flow, which includes database lookups to initialize the environment before running full inference via the `simulator.py` service.



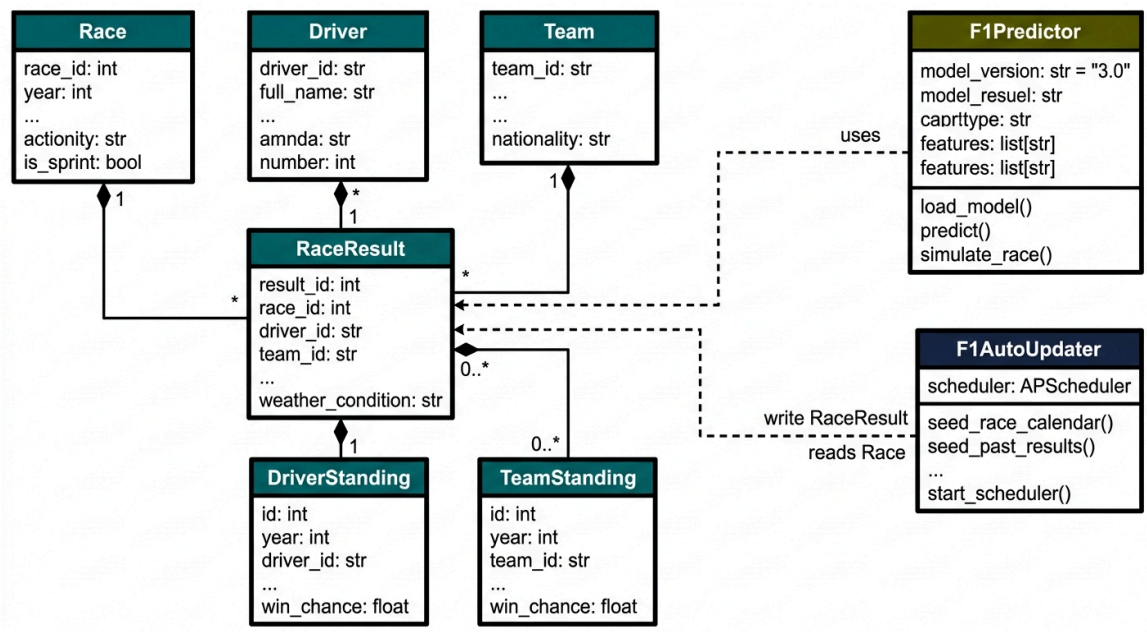
Sequence Diagram — Automated Data Ingestion

This diagram outlines the automated `APScheduler` pipeline, specifically showing how the `F1AutoUpdater` service fetches data from external APIs and uses `db_writer.py` to safely update the PostgreSQL database.



Class Diagram

This comprehensive UML Class Diagram shows the system's core structure. It organizes the database models (clustered on the left) separately from the specialized ML (`F1Predictor`) and Service (`F1AutoUpdater`) classes (on the right).



6. Operating Environment (5 points)

Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.

- **Python 3.12+** — backend API, FastAPI application, ML model execution, data ingestion pipeline, and database seeding scripts
- **Node.js 18+** — frontend build toolchain (Vite) and development server (port 5173)
- **Modern Web Browser (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+)** — client-side execution of the React application
- **PostgreSQL 16** — primary relational database server
- **FastAPI 0.135.1** — REST API framework served on port 8000
- **Uvicorn 0.41.0** — ASGI server for running the FastAPI application
- **APScheduler 3.11.2** — background job scheduler for automated 2026 race result ingestion, triggered approximately two hours after each race end datetime
- **SQLAlchemy 2.0.48** — ORM and database session management
- **Alembic 1.18.4** — database migration management
- **psycopg2-binary 2.9.11** — PostgreSQL adapter for Python
- **Pydantic 2.12.5** — response model definition and data validation
- **pydantic-settings 2.13.1** — environment variable loading and backend configuration management
- **scikit-learn 1.8.0** — ML model training and prediction (RandomForestClassifier v3.0, 12-feature input)
- **pandas 3.0.1** — data manipulation and feature engineering
- **joblib 1.5.3** — ML model serialization and deserialization (f1_winner_model_v3.pkl)
- **feedparser 6.0.12** — RSS feed parsing for the Newsroom page
- **Jupyter Notebook / JupyterLab 4.5.6** — ML workflow development and model training notebooks
- **React 18** — frontend UI library
- **TypeScript** — static type checking across all frontend code
- **Vite** — frontend build tool and development server
- **Tailwind CSS v4** — utility-first CSS framework for styling and responsive layout
- **React Router** — client-side routing between Dashboard, Simulator, Data Center, and Newsroom pages
- **React Query** — server state management and background data fetching
- **Axios** — HTTP client for frontend-to-backend communication with retry logic
- **Pytest 9.0+** — backend integration and unit testing framework
- **python-dotenv 1.2.2** — environment variable management for local development
- **Render** — cloud hosting platform; backend deployed as a web service, frontend deployed as a static site

7. Assumptions and Dependencies (5 points)

List any assumed factors (as opposed to known facts) that could affect the requirements stated in this document. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project.

Assumptions:

- Historical F1 race data from 2010 through 2025 is available, accurate, and sufficient for training a competitive prediction model.
- 2026 season race results will be accessible via the Jolpica F1 and OpenF1 APIs within a reasonable time after each race completes.

- Driver and team performance patterns contain enough signal year-over-year to produce predictions meaningfully better than random baseline.
- The Jolpica F1 API remains publicly available and free to use without authentication requirements throughout the project. Note: Jolpica deprecated its weather data endpoint during Increment 3; weather data is now sourced from Visual Crossing (historical backfill) and OpenWeatherMap (live forecasts).
- OpenWeatherMap remains available under its current free-tier terms for the duration of the project.
- Visual Crossing remains available under its current free-tier terms for historical weather backfill operations.
- The application will be accessed primarily via modern desktop browsers on 1920×1080 or larger displays; mobile support is implemented but not the primary evaluation target.
- Render free tier remains available and sufficiently stable for capstone demonstration purposes, acknowledging that cold starts may affect initial response times.

Dependencies:

- **Jolpica F1 API** — primary source for historical and current race schedules, results, driver standings, and constructor standings.
- **OpenF1 API** — supplementary real-time race data source for the automated ingestion pipeline.
- **OpenWeatherMap** — live and forecast weather data for race locations, used in the Simulator and Dashboard weather display.
- **Visual Crossing** — historical weather data used as a training feature for the ML model via one-time backfill scripts.
- **rss2json / feedparser** — RSS feed parsing for the Newsroom page, aggregating F1 news from multiple external sources.
- **scikit-learn 1.8.0** — machine learning library for model training and prediction (RandomForestClassifier v3.0).
- **joblib 1.5.3** — model serialization and deserialization for the trained prediction artifact (f1_winner_model_v3.pkl).
- **PostgreSQL 16** — primary relational database for all race, driver, circuit, prediction, and standings data.
- **SQLAlchemy 2.0.48** — ORM layer for all database interactions from the FastAPI backend.
- **APScheduler 3.11.2** — background scheduler for automated 2026 race result ingestion.
- **FastAPI / Uvicorn** — backend REST API framework and ASGI server.
- **pydantic-settings** — environment variable management and configuration validation in the backend.
- **React 18 / TypeScript / Vite** — frontend UI library, type system, and build toolchain.
- **React Router** — client-side navigation between Dashboard, Simulator, Data Center, and Newsroom pages.
- **React Query** — server state management and background data fetching in the frontend.
- **Axios** — HTTP client for frontend-to-backend API communication with interceptors and retry logic.
- **Tailwind CSS v4** — utility-first CSS framework for styling and responsive layout.
- **Render** — cloud hosting platform for both frontend static site and backend web service; free tier used for capstone deployment.
-